K.I.V.A.S. Stage 2

Video Tutorial Guide and Narration Scripts

Viewing guide

Start with Video 01 and continue in order. Use the embedded English captions as needed. When a prediction question appears, pause the video and try it before advancing. The activities use the diagrams and examples shown in the videos. Paper or a notes app is enough.

The visuals are from the K.I.V.A.S. Stage 2 PowerPoint. The scripts below reproduce the approved Markdown narration. Chapter timestamps are relative to the start of each exported video.

Production credit. The videos use ElevenLabs Brian narration with occasional spoken hesitations and light chuckles. The team should record significant AI assistance and its final review evidence in the assignment logs.

01 · Algorithms

Filename K.I.V.A.S_Video01_Algorithms_Brian.mp4
Duration and size 05:36 · 7.86 MB (7,856,167 bytes)
Learning goal Trace loops, compare linear and binary search, and follow a Bubble Sort pass.
Prerequisites Basic arithmetic and reading a list.
Related practice Study Guide I and Test Bank ALG OE-01–OE-04.
PowerPoint Slides 1–23.

Time	Chapter
00:00	Pip’s delivery problem
01:42	Variables and loop tracing
02:23	Linear and binary search
04:05	Bubble Sort and growing work

02 · Data Structures

Filename K.I.V.A.S_Video02_Data_Structures_Brian.mp4
Duration and size 05:10 · 7.73 MB (7,726,036 bytes)
Learning goal Choose structures for lookup, waiting order, hierarchies, and travel routes.
Prerequisites Video 01, especially indexing and sorted search.
Related practice Study Guide II and Test Bank DS OE-01–OE-04.
PowerPoint Slides 24–46.

Time	Chapter
00:00	Choosing a structure
00:26	Arrays and linked lists
01:46	Stacks and queues
02:51	Trees and campus graphs
04:42	Putting the structures together

03 · Connectionist AI

Filename K.I.V.A.S_Video03_ConnectionistAI_Brian.mp4
Duration and size 05:48 · 8.45 MB (8,454,349 bytes)
Learning goal Trace a forward pass and one weight update, then interpret evaluation metrics.
Prerequisites Basic multiplication, negative numbers, and percentages.
Related practice Study Guide III and Test Bank AI OE-01–OE-04.
PowerPoint Slides 47–71.

Time	Chapter
00:00	Learning signals and features
00:39	Neurons and forward passes
03:08	Loss and training
04:05	Validation, metrics, and responsible use

04 · Smart Campus Library

Filename K.I.V.A.S_Video04_Smart_Campus_Library_Brian.mp4
Duration and size 02:35 · 3.70 MB (3,698,970 bytes)
Learning goal Trace an exact library request through lookup, queueing, routing, and handoff. Explain where an optional recommendation fits.
Prerequisites Videos 01–03 or equivalent familiarity with search, queues, graphs, and prediction scores.
Related practice Study Guide III’s Smart Campus Library project and OE-04 from each Test Bank section.
PowerPoint Slides 72–81.

Time	Chapter
00:00	Library challenge
00:31	Worked request and route
01:00	Optional recommendation and failure responses
01:47	Practice and exit check

Narration · Video 01 · Algorithms

00:00 · Slide 1 · Can Pip run the library?

Pip has a request. Find copy twenty-four and deliver it to Gate. Another visitor is already waiting. Before Pip takes a step, what information does it need? Keep that question in mind. We'll finish the delivery together at the end.

00:14 · Slide 2 · Our learning route

First we'll trace the instructions Pip follows. Then we'll choose how to store the records, the waiting line, and the campus map. Finally, we'll see how Pip could make and test an optional recommendation.

00:27 · Slide 3 · Instructions a literal helper can follow

Imagine you tell Pip to put peanut butter on bread. Pip puts the closed jar on top of the bread. It followed a literal reading. To get the result we wanted, we need to say to open the jar, scoop some peanut butter, and spread it on the bread.

00:41 · Slide 4 · A task has a boundary

Let's trace a book return. The input tells us that book eighteen came back. Its record changes from on loan to available. The receipt is the output. Keeping those three things separate helps us tell what happened inside the system.

00:56 · Slide 5 · Computational thinking shapes the job

Break delivery into smaller jobs. Validate the request, locate the copy, and deliver it. For the route, keep paths and closures because they affect where Pip can walk. Once the procedure works, reuse it for the next delivery.

01:10 · Slide 6 · A legal first crossing?

The farmer can cross alone or take one passenger. Left alone together, the wolf eats the goat and the goat eats the cabbage. Which passenger should cross first? Pause and test what stays behind.

01:27 · Slide 7 · A solution can include a backward move

Take the goat across first. Return alone, take the wolf across, and bring the goat back. Take the cabbage across and return alone. Finally, take the goat across again. Bringing it back kept both banks safe while the cabbage crossed.

01:42 · Slide 8 · Variables and conditions

A variable holds a value we can update. Start count at zero, then set it to its current value plus one. A condition chooses an action. If the copy is available, Pip holds it.

01:53 · Slide 9 · Prediction: what does the loop print?

Start with a sum of zero. The loop visits two, four, and one in that order. Add each value to the running sum. Pause here and write the value after every step. What gets printed?

02:08 · Slide 10 · A trace shows the changing state

The sum starts at zero. Add two to get two, then four to get six, and then one to get seven. The program prints seven. A trace lets us see exactly where each value came from.

02:23 · Slide 11 · Linear search checks one item at a time

To find twenty-four, start at the left. Eighteen and seven fail the comparison. Twenty-four matches at index two. We count indices from zero, so index two is the third position.

02:36 · Slide 12 · A miss and a duplicate change the contract

If the target is missing, a scan must reach the end. If the task asks for the first match, stop at the first copy. If it asks for all matches, continue scanning and record each matching index.

02:48 · Slide 13 · Sorted order makes elimination possible

This row is sorted. The middle value is twelve and our target is fifteen. Every entry at twelve's position or farther left is at most twelve. That single comparison lets us discard indices zero through three.

03:04 · Slide 14 · Binary search keeps inclusive bounds

Low and high mark the first and last candidate positions. To get the middle, subtract low from high, halve the difference, round down, and add low back. Candidates remain as long as low is at most high.

03:17 · Slide 15 · Prediction: where does 13 go?

Try the same procedure with thirteen. Start with low at zero and high at six. Pause and write down the bounds and midpoint for each comparison until you find a match or run out of candidates.

03:33 · Slide 16 · Crossed bounds prove absence

At index three, twelve is too small, so low becomes four. At index five, eighteen is too large, so high becomes four. At index four, fifteen is still too large, so high becomes three. Low is now greater than high. Thirteen is absent.

03:52 · Slide 17 · Binary search for the first duplicate

The middle seven is a match, but an earlier seven might exist. Save its index and keep searching left. Here the next comparison moves us past the two, and we eventually find the first seven at index one.

04:05 · Slide 18 · Bubble Sort repairs adjacent disorder

Bubble Sort compares neighbors. Five is greater than one, so swap them. The row becomes one, five, four, two. Keep moving right and compare the values in their current positions.

04:16 · Slide 19 · Prediction: one complete pass

Restart from five, one, four, two. Compare the first pair, then the middle pair, then the last pair. Pause and trace each swap on the updated row. Where does five end up?

04:36 · Slide 20 · One settled value is progress

Five moves past one, four, and two, ending at the far right. The second pass puts two before four. A later pass with zero swaps confirms the row is sorted and lets us stop early.

04:48 · Slide 21 · Efficiency describes growing work

These labels describe how work grows as the input grows. A known array index takes constant work. Binary search repeatedly halves the candidates. A full scan grows with the number of items, while Bubble Sort can require quadratic work in the worst case.

05:05 · Slide 22 · Preparation changes the total cost

In this example, scanning costs eight units per query. Sorting costs twenty up front, then searching costs four per query. At five queries both plans cost forty. Starting with the sixth, the preparation pays off under these assumptions.

05:20 · Slide 23 · Next steps: algorithms

Before moving on, use the Study Guide sections shown on screen to revisit tracing, searching, and sorting. Try algorithm exercises O E zero one through zero four from the Test Bank. Trace an answer before checking it.

Narration · Video 02 · Data Structures

00:00 · Slide 24 · How should Pip store the same facts?

Pip has a record for each copy and a line of requests waiting to be served. A copy record and a place in line answer different questions. Let's choose a structure based on the operations each job needs.

00:13 · Slide 25 · A workload makes a choice defensible

If you know a position, you want direct indexed access. If people must be served in arrival order, you need a waiting line. If Pip needs to travel, the key information is which locations are connected.

00:26 · Slide 26 · Arrays locate a known index directly

Suppose the array starts at address one hundred and each slot is four units wide. Index four is four slots from the start. Four times four is sixteen, so that slot starts at address one hundred sixteen.

00:40 · Slide 27 · Insertion makes room before writing

To insert B between A and C, shift D right into the empty slot, then shift C right. Now place B in index one. Moving from the far end protects the entries we still need.

00:52 · Slide 28 · Length and capacity measure different things

Length counts entries in use. Capacity counts reserved slots. If there's a spare slot, appending can use it. When the array runs out of room, growth can involve allocating a larger array and copying the existing entries.

01:07 · Slide 29 · Linked lists follow next references

A linked list starts at its head. Each node stores a value and a reference to the next node. Follow those references one by one. A null reference marks the end.

01:18 · Slide 30 · Prediction: which link changes first?

We want to insert X after A and keep B and C in the chain. Which update should happen first? Pause and think about what A's next reference currently points to.

01:32 · Slide 31 · Safe splicing preserves the remainder

First set X's next reference to A's current next reference, B. Then point A to X. B and C stay reachable. To remove X, point A straight to whatever comes after X.

01:46 · Slide 32 · A stack exposes the most recent item

Push A, then B, then C. C is now on top. Pop removes and returns it. Peek lets you see the top without removing it. This is last in, first out.

01:58 · Slide 33 · Undo stores enough evidence to reverse a change

Suppose a copy moves from shelf B four to shelf C two. Save the copy's ID, old shelf, and new shelf in the undo record. When you undo that action, the old shelf tells you exactly what to restore.

02:13 · Slide 34 · Prediction: who is next in the queue?

Ada, Bo, and Cy are in line. Serve the front person, add Di at the back, and serve the front person again. Pause and work out who is now next.

02:30 · Slide 35 · A queue preserves waiting order

Ada leaves first. Di joins behind Cy. Bo leaves next. Cy is now at the front and Di is behind. A queue serves the earliest arrival still waiting.

02:39 · Slide 36 · Circular storage preserves FIFO

Start at slot two and read three entries. You get C, then D, then wrap to slot zero for E. The front index and count tell us the logical order, even when it wraps around the array.

02:51 · Slide 37 · Trees represent a hierarchy

R is the root. To reach C from R, go through A. That's two edges, so C has depth two. The longest downward path from R is also two edges, so R has height two.

03:03 · Slide 38 · A binary search tree adds an ordering rule

The root is eight. Every value in its left subtree is smaller than eight and every value on the right is larger. Apply the same rule at each node. To search, a comparison tells you which subtree to follow.

03:18 · Slide 39 · Traversal changes when a node is recorded

Preorder records a node before its children. Inorder records it between the left and right subtrees. Postorder records it afterward. Follow the diagram as you compare the three sequences. Here inorder lists the search-tree values in sorted order.

03:35 · Slide 40 · Graphs represent legal connections

The Library, Café, Lab, and Gate are nodes. Walkways are edges, and their labels tell us the walking time in minutes. Closing a walkway removes that connection from the routes Pip can use.

03:49 · Slide 41 · Breadth-first search uses a queue

Breadth-first search uses a queue to explore one layer at a time. Starting at the Library, discover the Lab and Café. When we process the Lab, we discover Gate. The discovery path uses the fewest edges.

04:02 · Slide 42 · Depth-first search uses a stack

Depth-first search follows one branch as far as possible. In this trace, we go from the Library to the Lab and then the Café. At the Café we're out of new neighbors, so we backtrack to the Lab and continue to Gate.

04:14 · Slide 43 · Prediction: which route is shortest?

One route uses two walkways and the other uses three. Pause and add the times for each route. Which has fewer edges? Which takes fewer minutes?

04:27 · Slide 44 · Fewest edges can take longer

Library to Lab to Gate takes nine plus two, or eleven minutes. Going through the Café takes two plus three plus two, or seven. If Café to Lab closes, the eleven-minute route is the remaining option.

04:42 · Slide 45 · One service combines several structures

Here's how the pieces work together. The sorted catalogue helps Pip find a copy by ID. The queue gives the order of requests. The weighted graph lets Pip compare legal routes by walking time.

04:55 · Slide 46 · Next steps: data structures

The Study Guide sections on screen cover the structures we just used. In the Test Bank, try data structures exercises O E zero one through zero four. For each problem, identify the operation before choosing the structure.

Narration · Video 03 · Connectionist AI

00:00 · Slide 47 · Can Pip predict a preference?

Finding copy twenty-four is an exact lookup. Suggesting a title someone might enjoy calls for a prediction. Let's look at how a system can compute a score and check whether it's useful.

00:12 · Slide 48 · Learning methods use different signals

Supervised learning uses examples with target labels. Unsupervised learning looks for patterns without target labels. Reinforcement learning uses rewards from taking actions. The difference here is the signal available during learning.

00:28 · Slide 49 · Features available at prediction time

Shared tags and past likes can be available when Pip scores a book. Whether the reader likes the suggestion may only become known afterward. That later feedback can be a training target for future examples.

00:39 · Slide 50 · Weights scale, bias shifts

Multiply each feature by its matching weight, add the products, then add the bias once. A weight changes how strongly an input contributes. The bias shifts the combined score.

00:51 · Slide 51 · An artificial neuron is a calculation

An artificial neuron takes those weighted contributions and the bias, then applies an activation rule. Think of it as two steps. Compute the combined value first, then transform it into the neuron's output.

01:05 · Slide 52 · Activation changes the output rule

Identity passes the value through. ReLU, pronounced ree-loo, outputs zero for negative inputs and keeps positive inputs as they are. The step rule outputs one when the value is zero or above, and zero otherwise.

01:19 · Slide 53 · A network follows dependencies

Both hidden units use both inputs. Their outputs then feed the final unit. Compute the two hidden values first so the final calculation has everything it needs.

01:30 · Slide 54 · Prediction: how many parameters?

Count the parameters in the network. Each connection has a weight and each computed unit has one bias. Pause and count weights and biases separately before adding them together.

01:45 · Slide 55 · Nine parameters, reusable across examples

The two hidden units have two weights and a bias each, for six parameters. The output unit has two weights and a bias, for three more. That makes nine. We reuse those same parameters on new examples.

01:59 · Slide 56 · Nonlinearity changes what layers can express

If we substitute two x plus one into the next identity calculation, the whole chain simplifies to six x plus seven. Put ReLU in the hidden layer, and negative hidden values are clipped to zero. Now the calculation can behave differently on different parts of the input range.

02:16 · Slide 57 · The hidden layer computes two views

Set the inputs to two and one. The first hidden unit gets two minus one, which is one. The second gets half of two, plus one, minus one, also one. ReLU keeps both values.

02:28 · Slide 58 · Prediction: finish the forward pass

Both hidden outputs are one. Multiply the first by two, add the second, then subtract point five. Pause and work out what the identity output returns.

02:44 · Slide 59 · A score needs a declared interpretation

The output is two point five. This toy decision rule flags a score of two or higher, so the request gets a flag. The threshold is what turns the numerical score into that decision.

02:55 · Slide 60 · One changed input can silence a hidden unit

Now the inputs are one and two. The first hidden unit gets a negative value, so ReLU makes its output zero. The second outputs one point five. After the output bias, the final score is one.

03:08 · Slide 61 · Loss measures a chosen kind of error

Pip predicts a four-minute wait and the real wait is six minutes. The prediction error is minus two. Squaring it gives four. Loss puts a number on the kind of error the training procedure is trying to reduce.

03:21 · Slide 62 · Training adds a parameter update

Start with a forward pass to get a prediction. Compare it with the target to calculate loss. Then use gradients to guide an update to the parameters. Repeat with more training data.

03:32 · Slide 63 · Prediction: one weight update

The weight starts at one. The gradient is negative eight and the learning rate is point one. Subtract the learning rate times the gradient. Pause and calculate the new weight, then multiply it by the input two.

03:49 · Slide 64 · The update improves this example

Subtracting negative point eight gives one point eight. The new prediction is three point six. Squaring the difference from four gives point one six. In the batch example, twelve cases split into batches of four give three updates per epoch.

04:05 · Slide 65 · When training loss misleads

Follow both curves. Training loss keeps falling, while validation loss starts rising after epoch two. Of the checkpoints shown, epoch two has the lowest validation loss, so it's the choice supported by that split.

04:18 · Slide 66 · Data splits answer a future-use question

Use training data to update parameters. Use validation data to choose things like a checkpoint. Once the choices are fixed, use test data to evaluate the final model.

04:30 · Slide 67 · Prediction: what do the errors tell us?

Eight urgent requests were flagged and two were missed. Four routine requests were flagged and eighty-six were ignored. Pause and count how many cases were correct, how many urgent requests there were, and how many flags Pip raised.

04:47 · Slide 68 · Different metrics expose different costs

Accuracy is ninety-four correct out of a hundred. Recall is eight detected out of ten urgent cases. Precision is eight urgent cases out of twelve flags, or about sixty-six point seven percent. The denominators tell us which question each number answers.

05:03 · Slide 69 · Applications still need a defined target

Pixels can become arrays of numbers. Text can be converted into a representation that captures some of its information. For recommendations, define what a useful ranking should predict, then rank the books that are eligible to be offered.

05:18 · Slide 70 · Responsible AI in the library

Ask permission before using reading history. Look beyond one overall score. In this example, recall is eighteen out of twenty in one group and ten out of twenty in another. Give readers a way to report and correct a problem.

05:32 · Slide 71 · Next steps: connectionist AI

Use the Study Guide sections on screen to review forward passes, training, and evaluation. Try the AI exercises O E zero one through zero four. For each one, show the calculation or the evidence behind your choice.

Narration · Video 04 · Smart Campus Library

00:00 · Slide 72 · The Smart Campus Library challenge

Back to our opening request. Copy twenty-four is available and the first reader wants it delivered to Gate. They decline personalization. The second reader wants copy twelve at the Library counter. Copy thirty-seven is already on loan.

00:14 · Slide 73 · Team challenge: design the service

Sketch the catalogue, waiting line, campus map, and request status. Pause and trace the first request, then the second. Show what still happens when the first reader declines a personalized recommendation.

00:31 · Slide 74 · The request trace keeps facts separate

The catalogue finds copy twenty-four at index one. Put the copy on hold for the first request while the second waits. At handoff, mark the first request delivered and update the copy's status to on loan.

00:45 · Slide 75 · The current map decides the route

With all paths open, go through the Café and Lab for seven minutes. If the Café-to-Lab path closes, go straight to the Lab and then Gate for eleven. If Lab-to-Gate closes as well, Gate is unreachable on this map.

01:00 · Slide 76 · An optional book suggestion

For a reader who opts in, the example model scores two eligible books. A shares two tags and has three prior likes. B shares three tags and has zero prior likes. Under the example weights, A ranks higher. The score supports a suggestion.

01:17 · Slide 77 · Prediction: which failure needs which response?

What should Pip say if the copy is already on loan? What if Gate becomes unreachable before departure? And what if the recommendation model is unavailable? Pause and decide what should happen in each case.

01:34 · Slide 78 · A useful system reports what actually happened

If the copy is on loan, report it unavailable. If the route disappears before departure, release the hold and report the route problem. If the model is down, carry on with the exact library service.

01:47 · Slide 79 · Next steps: build an auditable blueprint

Build your own trace for the Smart Campus Library project. Include the starting records, the order of requests, and the changes after each decision. Then try the O E zero four exercise from each Test Bank section.

02:01 · Slide 80 · Exit check: explain one changed condition

Pick one change: a duplicate ID, a walkway closure, or a new reader. Pause and explain what assumption changed. What extra step in the trace, or what extra evidence, would you now need?

02:18 · Slide 81 · What made Pip useful?

Pip can handle the delivery because the procedure is explicit, the records and map track what's happening, and the handoff confirms the result. For readers who opt in, a tested prediction can add a suggestion. Every part of the service has a job you can trace.